用户与鉴权的业务边界

gomall · 注册 / 双 token / RBAC / admin bootstrap / 三层鉴权路线图

---

gomall · 业务侧 deck

今日大纲

业务困局：鉴权不止是” 登录拿 token”

注册 + 登录 + 双 token：第一道业务边界

RBAC：30s 缓存为什么业务能接受

admin bootstrap + 三层边界路线图

业务场景：补券 / 业务码 / 客服话术

双 token 失效 / 强制下线 / 多端隔离

业务困局：鉴权不止是”登录拿token”

鉴权是商业模式的边界

- 一个电商，凡是和”钱”挂钩的接口，背后都是一句话：**这个请求是谁发的，他能做什么**。
- gomall 把这句话拆成三件事：
  - **身份**：注册 / 登录 / token 续期——客户在不在。
  - **角色**：user / merchant / admin ——客户能看见什么。
  - **边界**：路由分组 + middleware 链——客户做不到什么。
- 任何一件做错，损失都是真金白银：身份错则越权下单，角色错则客服把全场红包发给自己，边界错则 admin 接口暴露给 C 端。

routes/router.go:34–45 公开 vs authed；:148–155 admin 子组

gomall 现在的三层墙（实际只立起来两层）

- 公开层：能匿名访问，挂 HTTP cache，不挂任何 auth。
- authed 层：所有 C 端用户登录后能用，挂 `AuthMiddleware`。
- merchant 层：**当前还没有独立 group**，商家身份借 user 表 Role 字段表达。
- admin 层：在 authed 内嵌一层 `RequireRole("admin")`。

routes/router.go:39–45 公开；:47–48 authed group；:148–150 admin sub-group

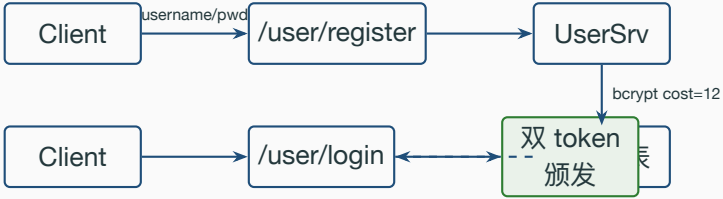
业务问题清单 + 本 deck 的回答顺序

1. 注册 + 登录链路，密码加密 / 邮箱验证业务边界画在哪。
2. access 24h / refresh 10d 这两个数字怎么定，怎么影响留存。
3. RBAC 为什么敢上 30s 内存缓存。
4. admin bootstrap 这个” 一次性接口” 为什么必须存在。
5. C 端 / merchant / admin 三层边界，merchant 还差什么。
6. 客服补券走哪条路？为什么用户自助补券是黑产温床。
7. 业务码 `30001` / `30002` / `30005` 对应的客服话术。

鉴权是**业务对” 信任” 的一次明确表态**：信谁，信多久，信他能干什么。

注册 + 登录 + 双 token：第一道业务边界

注册 / 登录 / 颁发 token 的时序



routes/router.go:34–35 路由；

service/user.go:38–82 注册；*:85–121* 登录；*repository/db/model/user.go:28 PassWordCost=12*

密码是怎么存的：bcrypt cost=12

- **bcrypt** 自带 salt，单向，不可还原。重置密码只能让用户重新设置。
- `PassWordCost = 12`：单次哈希 ~250ms（M 系列 Mac）。
  - 业务收益：撞库脚本一秒只能猜 4 次，10 位密码组合在合理时间内不可穷举。
  - 业务代价：登录接口 RT 多 250ms，C 端可接受。
- 注册存哈希，登录拿明文重算 hash 再 compare。**后台永远拿不到明文**，被拖库后撞库代价巨大。
- 找回密码必须走**邮箱验证 + token 写新摘要**：`EmailClaims` 携带 bcrypt 摘要而非明文，TTL **15min** 远短于 access。

repository/db/model/user.go:34–48 SetPassword / CheckPassword；service/user.go:56 注册调用；pkg/utils/jwt/jwt.go:95–134 EmailClaims

双 token 续期的状态机

- **access valid**：`AuthMiddleware` 顺便**续发**新 access + refresh，写回 header + cookie。
- **access 过期 / refresh 有效**：拿 refresh 重发一对新 token，用户无感知。
- **两者都过期**：返回 `30001`，C 端跳登录页。

pkg/utils/jwt/jwt.go:67–93 ParseRefreshToken；middleware/jwt.go:59 SetToken；consts/user.go:31–32 24h/10d

24h / 10d 这两个数字的业务依据

参数	取值	业务依据
access 短时窗	24h	单日活跃无需重登；token 泄露最长 24h
refresh 长时窗	10d	周末 + 黄金周回流用户仍能静默续期
邮箱 token	15min	邮件链路被截短 TTL 反制
RBAC 缓存	30s	提权 / 降权延迟可接受

- 10d 是**留存指标的妥协**：app 端” 打开就在线” → 用户不流失。
- access 24h 不能更长：移动端被刷机 / 偷设备时，攻击窗口必须有限。
- 不做 7d：周中休假 + 周末” 双周回归” 用户会被强制重登，体验差。

consts/user.go:31–32 AccessTokenExpireDuration / RefreshTokenExpireDuration

代码片段 1：AuthMiddleware 主链路

Listing 1: *middleware/jwt.go:16–64 AuthMiddleware*

```
accessToken := c.GetHeader("access_token")
refreshToken := c.GetHeader("refresh_token")
if accessToken == "" {
   code = e.InvalidParams
   c.JSON(200, gin.H{"status": code, ...})
   c.Abort(); return
}
newAccessToken, newRefreshToken, err :=
   util.ParseRefreshToken(accessToken, refreshToken)
if err != nil { code = e.ErrorAuthCheckTokenFail }
...
SetToken(c, newAccessToken, newRefreshToken)
c.Request = c.Request.WithContext(
   ctl.NewContext(c.Request.Context(), &ctl.UserInfo{Id: claims.ID}))
```

逐行讲解 1：AuthMiddleware

- `GetHeader` 两个 token ——C 端 SDK 约定都从 header 拿，不解析 cookie（避免与第三方网关 cookie 同名冲突）。
- `accessToken == ""` 直接 `400 InvalidParams`，**不是 30001**。区分” 完全没带” vs ” 带了但错”，便于客服判断是不是 SDK bug。
- `ParseRefreshToken` 一次出双 token：每次请求都续期，access 永远 24h 起跳。
- `SetToken` 同时写 `header + cookie + SameSiteLax`：兼容 SPA 和老 web 多窗口。
- 潜在坑：续期 token 写 header 后，C 端必须**用最新 header 覆盖本地存储**，否则下一次还是老 token。
- 可改进：当前 `ParseRefreshToken` 把”access 没过期” 也当作续期信号，浪费一次 HS256 签名。可加” 剩余 < 1h 才续” 的判断。

middleware/jwt.go:20–62

双 token 不查黑名单 + 鉴权链路开销

场景	RPS	p95
/ping （基线）	64,254	3.51ms
/orders/list （带 Auth）	58,406	5.00ms

鉴权链路开销 $\approx 1 - 58406/64254 = 9.1\%$，

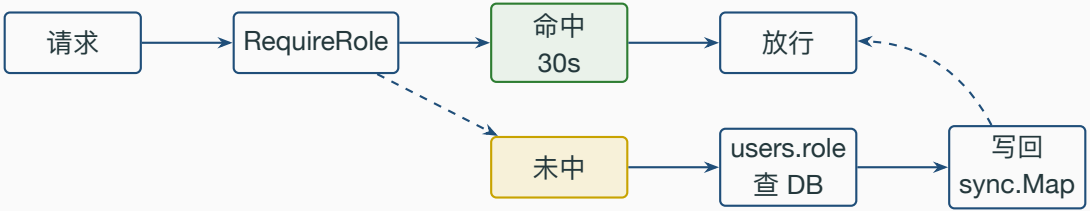
敢挂 authed 全组。HS256 纯 CPU 无 IO。

- 当前**不维护 token 黑名单**：登出 = 客户端丢 token，服务端不主动作废。
- 后果：被偷设备最多多用 24h；改密码后老 access 仍能用 24h。
- 敢这么做的前提：token 仅控访问，**不直接控钱**；提现 / 大额支付有二次验证。
- 路线图：登出 / 改密码事件写 Redis 黑名单 set（TTL = access 剩余），鉴权链路多一次 `SISMEMBER`。

stressTest/REPORT.md 链路压测表行 *1, 4*；*middleware/jwt.go:32–57* 当前无黑名单

RBAC：30s 缓存为什么业务能接受

RBAC 30s 内存缓存命中链路



middleware/rbac.go:24 roleCacheTTL=30s；:28–45 lookupRole；:48–86 RequireRole

代码片段 2：lookupRole + 30s sync.Map 缓存

```go
22 var (
23    roleCache sync.Map
24    roleCacheTTL = 30 * time.Second
25 )
26 func lookupRole(ctx context.Context, userId uint) (string, error) {
27    if v, ok := roleCache.Load(userId); ok {
28       e := v.(roleCacheEntry)
29       if time.Now().Before(e.expires) { return e.role, nil }
30    }
31    u, err := dao.NewUserDao(ctx).GetUserById(userId)
32    if err != nil { return "", err }
33    role := u.Role
34    if role == "" { role = "user" }
35    roleCache.Store(userId, roleCacheEntry{role, time.Now().Add(roleCacheTTL)})
36    return role, nil
37 }
```

middleware/rbac.go:22–45

逐行讲解 2：lookupRole

- `sync.Map`：**读多写少**的典型场景，比 `RWMutex+map` 在高并发读下吞吐更高。
- `Load` 走 ConcurrentMap fast-path；命中后**零锁**返回。
- `role == ""` 的兜底是**业务安全网**：早期老用户 Role 字段是空字符串，强制视为最低权限 user。
- 写回不加锁：sync.Map.Store 内部 atomic，并发覆盖也只是同值，业务无影响。
- 潜在坑：进程重启缓存清零，第一波 admin 请求会全部回源 DB。单实例影响很小。
- **为什么不用 Redis**：本机 sync.Map 是**零 RTT**，鉴权延迟低于 1ms；Redis 反而引入网络依赖。

30s 在业务上意味着什么 + 显式失效兜底

- 提权场景：管理员把客服 A 升为 admin → 最长 30s 后享受 admin 权限。
- 降权场景：admin 误授权后回收 → 误授权用户最长 30s 内仍能继续操作。
- 30s 误授权窗口能干坏事吗?
  - admin 当前接口（users / promote / backfill）不会瞬间造成资金损失。
  - 真正控钱的接口（提现 / 退款）尚未挂在 admin 子组，无 30s 资金风险。
- 必须零 TTL 的场景（封禁恶意 admin），用 `InvalidateRoleCache(userId)` **显式失效**。
- 选型 = **最终一致 + 关键路径显式失效**：99% 走 0ms 缓存命中，1% 走 Invalidate 立即生效。

middleware/rbac.go:88–91 InvalidateRoleCache；service/admin.go:47–62 PromoteToAdmin 写完即 Invalidate

admin bootstrap + 三层边界路线图

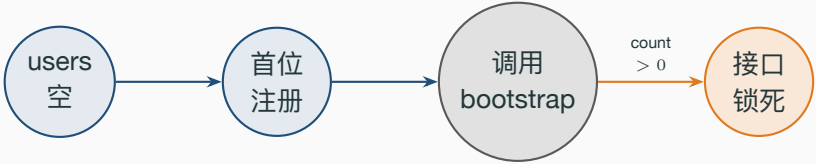
冷启动悖论

- 上线一个全新 gomall 实例，users 表是空的。
- 想给”运维大佬”开 admin 权限，必须调 `/admin/users/promote`。
- 但 `/admin/...` 子组的中间件链是 `Auth + RequireRole("admin")`。
- **此时一个 admin 都没有，谁也进不去 admin 子组** → 没人能把别人提升为 admin。

> 这就是 admin bootstrap 必须存在的原因：**先有第一个 admin，后面 RBAC 才能转起来。**

routes/router.go:146 admin/bootstrap 仅挂 authed，无 RequireRole；service/admin.go:66–80 BootstrapPromoteSelf

冷启动状态机



- **核心校验**：`BootstrapPromoteSelf` 在执行前 `SELECT count(*) WHERE role='admin'`。
- count $= 0$：把当前调用者升为 admin。
- count > 0：返回错误”系统已存在 admin”，**接口从此永久不可用**。
- 路由层做法：`authed.POST("admin/bootstrap")`，仅在 `AuthMiddleware` 之下，但**不**挂 `RequireRole`。

service/admin.go:72–78 count 校验；routes/router.go:146

代码片段 3：BootstrapPromoteSelf 一次性校验

Listing 2: *service/admin.go:66–80 BootstrapPromoteSelf*

```go
func (s *AdminSrv) BootstrapPromoteSelf(ctx context.Context) error {
    u, err := ctl.GetUserInfo(ctx)
    if err != nil { return err }
    db := dao.NewUserDao(ctx).DB
    var count int64
    if err := db.Model(&model.User{}).
        Where("role = ?", model.RoleAdmin).
        Count(&count).Error; err != nil {
        return err
    }
    if count > 0 {
        return errors.New("系统已存在 admin，禁止使用 bootstrap 接口")
    }
    return s.PromoteToAdmin(ctx, u.Id)
}
```

逐行讲解 3：BootstrapPromoteSelf

- `ctl.GetUserInfo` 取的是 `AuthMiddleware` 写进 ctx 的 user id ——**调用者必须已登录**。匿名打 bootstrap 会先被 AuthMiddleware 截掉。
- `count(*) WHERE role='admin'`：业务**自检**。`users.role` 有 index，实际是 range scan，毫秒级。
- `count > 0 return err`：这一行就是**接口" 自锁" 机制**。
- `PromoteToAdmin`：复用提权逻辑，触发 `InvalidateRoleCache` 立即生效。
- 潜在并发坑：两人同时打 bootstrap，count=0 都通过 → 两人都被提为 admin。**第一次部署窗口**通常只有运维一人，可接受。真要修：`SELECT ... FOR UPDATE` 或表级 advisory lock。

C 端 / merchant / admin 三层对比

角色	路由位置	中间件链	业务能力
匿名 user	v1 顶层	仅全局桶	浏览商品
C 端 user	authed 组	+ AuthMiddleware	下单 / 抢券
merchant	暂无	—	**路线图**
admin	admin 子组	+ RequireRole	提权 / 后台

- C 端 vs admin 边界靠 `RequireRole("admin")` 拦下。
- merchant 还没独立 group ——商家身份只是 user 表 Role 字段的占位。
- product/create 现在在 authed 组：任意已登录用户都能调，**没有商家校验**。

routes/router.go:47–48 authed；:61–63 product/create 待迁；:148–155 admin

merchant group 的路线图：路由 + DAO 双层

1. authed 内增设 `merchant := authed.Group("/merchant")`，挂 `RequireRole("merchant","admin")`。
2. 商品 CRUD / 库存 / 订单导出全部迁过来。
3. users 表加 `shop_id` 外键，DAO 层每个 query 强制带 `shop_id = current_user.shop_id`。

> RBAC 解决”**他能不能进这个路由**”；shop_id 解决”**他能不能动这条记录**”。两者缺一不可。

repository/db/model/user.go:23 Role 字段已就绪；routes/router.go:61–63 待迁移

业务场景：补券 / 业务码 / 客服话术

客服补券的路径选择

方案	路径	黑产风险	审计
A. 用户自助补	C 端调 /coupon/claim	**高**	无
B. 客服走 admin	admin 后台接口	低	有

- **A 不行**：用户自助意味着 token 持有者就能领，黑产**万号脚本** 1 分钟领光所有券。
- **B 是标准**：补券必须由**有 admin 权限的客服**发起，每次请求都**留 operator_id 进审计日志**。
- gomall 当前 admin 子组只有 promote / backfill，**补券接口待补**。
- 路线图：`admin.POST("coupon/grant", ...)` 接收 target_user_id + reason，写 admin_audit 表。

routes/router.go:148–155 admin 子组当前只有 3 个接口；deck 10 SlidingWindow 防自助补券黑产

业务码 + 客服话术（鉴权三种姿势）

码	含义	客服话术
400 InvalidParams	token 没带 / 格式错	” 请更新最新版 app 再试”
30001 ErrorAuthCheckTokenFail	token 解析失败	” 请重新登录”
30002 ErrorAuthCheckTokenTimeout	双 token 都过期	” 您已超过 10 天未登录，请
30005 ErrorAuthInsufficientAuthority	角色不够	” 该操作需要管理员权限”

- 400 vs 30001：客服区分”**SDK bug** vs **token 真坏**”。
- 30001 vs 30002：用户视角都是” 要重登”，但客服能区分” 账户安全事件 vs 正常长闲置”。如果某用户 30001/30002 比例异常，可能是被换设备登录，触发安全审计。
- 30005：admin 接口必备——普通用户摸 admin 时给清晰提示。

pkg/e/code.go:35–58；middleware/jwt.go:24,38,51 三处 c.JSON(200,...)；middleware/rbac.go:77

鉴权路线图（按优先级）

1. **P0**：登出 / 改密码 token 黑名单（Redis set + TTL）。
2. **P0**：admin 操作审计表 admin_audit（operator / target / reason / ts）。
3. **P1**：merchant 独立 group + DAO 层 shop_id 强制隔离。
4. **P1**：bootstrap 接口加 advisory lock，防并发提权。
5. **P2**：admin 子组的” 补券 / 退款 / 封号” 标准接口。
6. **P2**：access 续期改” 剩余 < 1h 才续”，省一次 HS256。

鉴权不是” 加一个 if”，是**用最少的代码代价表达最严的业务边界**。

middleware/jwt.go：待加黑名单；*service/admin.go*：待加 *advisory lock*；*routes/router.go*：待加 *merchant group*

攻击面：登录接口要扛三种黑产打法

攻击类型	特征	单点拦截手段
撞库 (credential stuffing)	泄露库扫所有用户	失败限频 + 二次验证
密码喷洒 (spraying)	弱密码扫万人	全局账号失败计数
账号枚举	错误码差异爆破	错误码归一 / 时序对齐

- gomall 当前登录仅 bcrypt + 业务码 `10005`，**无失败限频**。
- 错误码上 `10003`（用户不存在）vs `10005`（密码错）可被批量枚举。
- 黑产场景：3000 IP 代理池 × 5 RPS = **15k RPS**，bcrypt 直接拖死。

service/user.go:85–121 login 当前无失败限频；*pkg/e/code.go:10003 vs 10005* 错误码差异是枚举漏洞

撞库防御三段式：限频 + 验证码 + 设备指纹

- **第一层**：IP 维度滑动窗口限频，复用 deck 10 `SlidingWindow`。
- **第二层**：账号维度连续失败计数，5 次失败强制带验证码。
- **第三层**：15min 内累计 10 次失败 → 冻结 30min，直接返回 `10005` 不消耗 bcrypt。
- bcrypt cost=12 每次 250ms，账号冻结后**省的是 CPU 不是用户体验**。

repository/cache/ratelimit/sliding_window.go 可直接复用；*service/user.go:85* 是接入点

代码片段 4：登录失败计数 + 强制验证码门槛

```go
const (
    loginFailKeyPrefix = "auth:login:fail:"
    loginFailThreshold = 5 // 15min 内
)

func (s *UserSrv) preflight(ctx context.Context, name, cap string) error {
    fails, _ := cache.RedisClient.Get(ctx, loginFailKeyPrefix+name).Int()
    if fails >= loginFailThreshold && cap == "" {
        return ErrCaptchaRequired // 业务码 10011
    }
    if fails >= loginFailThreshold {
        if ok, _ := captchaVerify(ctx, cap); !ok { return ErrCaptchaInvalid }
    }
    return nil
}
```

路线图：*service/user.go login* 入口前置校验

逐行讲解 4：preflight + 失败计数器

- `loginFailKeyPrefix`：key 设计成 `auth:login:fail:<username>`，TTL = 15min 滑动窗口。
- `Get(...).Int()`：Redis miss 直接返回 0，无需额外判空。
- `fails ≥ 5 && captcha == ""`：前端拿到 **10011** 直接弹 hCaptcha。
- 登录成功后必须 `Del(loginFailKeyPrefix+name)`，否则用户输错一次就锁 15min。
- 设备指纹路线图：UA + canvas hash + 时区组成 `deviceId`，跨账号失败 ≥ 3 直接拉黑。

config/redis.go cache.RedisClient 已就绪；deck 10 SlidingWindow 提供窗口算法

双 token 失效 / 强制下线 / 多端隔离

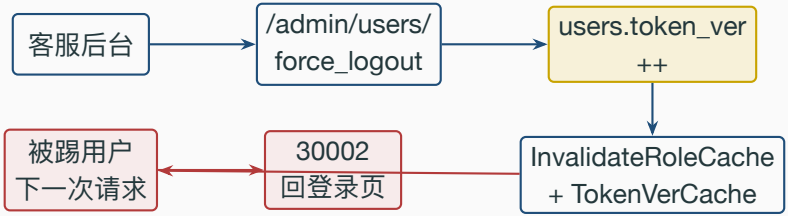
token 黑名单方案：版本号 vs SET

方案	实现	代价
A. Redis SET 黑名单	SISMEMBER `jti`	每请求 1 RTT，TTL 难管
B. 用户 token 版本号	users.token_ver，签进 claims	每请求查 DB / 本地缓存
C. 用户最早签发时间戳	users.token_min_iat，对比 iat	改密码只写一次

- A 按 token 粒度（被偷场景），`jti` 数随活跃用户线性增长。
- B/C 按用户粒度（改密码 / 强制下线），改一次字段全部作废。
- gomall 推荐 **B+C 合并**：版本号 ++ 即生效，本地 sync.Map 命中 0ms。

repository/db/model/user.go:14 待加 TokenVer；pkg/utils/jwt/jwt.go:18–22 Claims 加字段

客服强制下线时序



- 写 `token_ver`：单条 UPDATE 行锁，毫秒级返回。
- 失效本地缓存：与 RBAC 共用 `InvalidateRoleCache` 模式，按 userId 删本地 sync.Map 槽。
- 多实例场景：写 Redis pub/sub 让所有实例 Del 本地缓存槽，最终一致延迟 < **100ms**。

service/admin.go:47–62 PromoteToAdmin 已是模板；路线图：admin.go 加 ForceLogout(userId)

refresh 被偷的恢复路径

- **识别**：同 userId 的 refresh 出现在 2 个不同 IP / device → 钓鱼信号。
- **触发**：每次登录 / 刷新写 `userId` → `deviceId` HSET，device 数 > 5 触发风控。
- **处置**：`token_ver++` 全端下线，C 端跳登录页 + 短信 / 邮件二次验证。
- **兜底**：每次发 access 时推 `security_event` 到 IM / 邮件，让用户主动报警。

路线图：*repository/cache/security* 新增 *device* 集合；*jwt.go SetToken* 后写一次

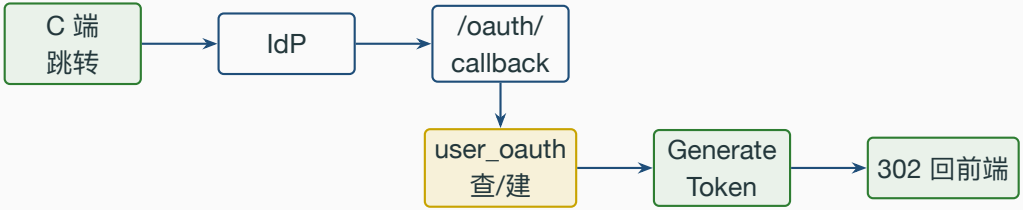
第三方登录接入对照

渠道	协议	关键字段	业务难点
微信 (开放平台 / 小程序)	OAuth2 变体	unionid / openid	账号合并
Google Sign-In	OIDC	sub / email	邮箱冲突
Apple Login	OIDC	sub / relay 邮箱	relay 去重
GitHub / 企业 SSO	OAuth2/SAML	login / orgs	角色映射

- gomall 当前只有用户名密码，第三方登录从 0 接入。
- 数据模型加 `user_oauth` 表：`(provider, external_id)` → `user_id`。
- **邮箱已注册**场景：合规要求二次验证，不直接合并。

路线图：*repository/db/model/user_oauth.go*；*service/oauth.go*；*router.go* 加 */oauth/callback*

第三方登录链路：把 OAuth 收敛回 gomall 双 token



- OAuth 拿 `external_id` 后**不直接颁 token**，先查 / 建 `user_oauth`。
- 后续接口仍走双 token + AuthMiddleware，IdP 不参与每次鉴权。
- `external_id` 已绑别人 → 拒绝，返回 `ErrorExistUser`。

pkg/utils/jwt/jwt.go:25–52 GenerateToken 复用；路线图：service/oauth.go BindOrCreate

bcrypt cost 调优：为什么是 12 不是 10 也不是 14

cost	单次耗时 (M1)	1 核 / 秒	1k 登录耗 CPU
10	~60ms	16 次	60s
12	~250ms	4 次	250s
14	~1.0s	1 次	1000s

- cost=10 太弱：消费级 GPU 一秒穷举几千候选，2026 标准 ≥ 12。
- cost=14 太重：登录多 1s，1k QPS 吃满整机 CPU。
- cost=12：250ms 在 300ms 感知阈值内；穷举 10 位数字 $\approx$ 80 年。
- bcrypt 内置 16 字节 salt，拖库后 rainbow table 失效。每 2–3 年 cost++ 一次。

repository/db/model/user.go:28 PassWordCost=12；:34–48 SetPassword/CheckPassword

代码片段 5：cost=12 与未来无痛升级路径

```go
const PassWordCost = 12

func (u *User) SetPassword(pwd string) error {
   b, err := bcrypt.GenerateFromPassword([]byte(pwd), PassWordCost)
   if err != nil { return err }
   u.PasswordDigest = string(b); return nil
}

func (u *User) CheckPassword(pwd string) bool {
   if bcrypt.CompareHashAndPassword(
      []byte(u.PasswordDigest), []byte(pwd)) != nil { return false }
   if cost, _ := bcrypt.Cost([]byte(u.PasswordDigest)); cost < PassWordCost {
      _ = u.SetPassword(pwd); _ = saveDigest(u) // 顺便升 cost
   }
   return true
}
```

repository/db/model/user.go:28–48 + 升级钩子

为什么 gomall 选纯 JWT，不走 session cookie

维度	Server Session	JWT (双 token)
鉴权 RT	读 Redis 1 RTT	解 HS256 零 RTT
失效粒度	删 session $O(1)$	黑名单 / token_ver
水平扩缩	共享存储瓶颈	无状态，加机器即可
跨域 / 多端	cookie SameSite	header 跨域无障碍

- gomall 优先：**鉴权延迟 + 水平扩展 + 多端** → JWT。
- 代价：失效慢，用 `token_ver` + 黑名单兜底关键路径。
- 反例：银行类业务选 session 更合理，失效粒度比延迟重要。

middleware/jwt.go 全文 64 行无 Redis 依赖；session 方案需要每请求 Get(sid)

多端同账号：app / web / 小程序 token 隔离

- 当前 gomall 所有端共享同一对 token：app 登录 web 自动登录。
- 业务影响：web 改密码 → app 上 token 仍能用 24h。
- 路线图：`Claims` 加 `client_type`（app / web / mp / pad），按 `(userId, client_type)` 颁发独立 token。
- 副作用：refresh 必须保留 client_type；强制下线粒度更精细；device 数风控按 client_type 分桶。

pkg/utils/jwt/jwt.go:18–22 Claims 待加 ClientType；service/user.go:85–121 login 待区分入口

审计日志：admin 操作必须留痕

- 当前 admin 3 个接口（promote / backfill / users），**无审计写入**。
- 合规字段：operator_id / target_id / action / reason / before / after / ts / ip / UA。
- 路线图：`admin_audit` 表 + `AuditMiddleware` 挂 admin 子组自动入表。
- 索引：`(operator_id, ts)` 客服自查；`(target_id, ts)` 投诉反查。
- 不允许 DELETE，按月分区，异地冷备 7 年。审计是补券 / 强制下线的前置依赖。

routes/router.go:148–155 admin 子组待加 AuditMiddleware；service/admin.go 所有写操作待加审计

鉴权链路涉及的全部业务码 + 客服对照

码	含义	客服话术
400 InvalidParams	token 缺失 / 格式错	升级 app 重试
10003 ErrorNotExistUser	用户名不存在	用户名或密码不正确
10005 ErrorNotComparePassword	密码错	用户名或密码不正确
10011 ErrCaptchaRequired (路线图)	失败过阈值	请完成人机验证
30001 ErrorAuthCheckTokenFail	token 解析失败	请重新登录
30002 ErrorAuthCheckTokenTimeout	双 token 都过期	已超 10 天未登录
30003 ErrorAuthToken	token 字段不匹配	内部错误，请重试
30005 ErrorAuthInsufficientAuthority	角色不够	需要管理员权限

SOP：10003 / 10005 对外话术一致（防枚举），对内保留具体码用于审计。

pkg/e/code.go:10001–10011, 30001–30010；路线图：*10011* 新增

鉴权链路容量规划：1k / 10k / 100k QPS 怎么撑

目标 QPS	access 解析 CPU	RBAC DB QPS (miss)	Redis RTT
1k	单核 12%	≈ 33	0.3ms p99
10k	4 核 30%	≈ 330	0.5ms p99
100k	32 核 60%	≈ 3.3k	1.5ms p99

- 压测基线：authed 链路 **58,406 RPS / p95 5.0ms**，单机已扛 50k+。
- 10k QPS 内单实例即可；100k 横向扩 4 实例 + 本地缓存命中 > 99%。
- **瓶颈不在鉴权**：下单 / 支付 / 库存先到瓶颈，鉴权剩 60%+ 余量。

stressTest/REPORT.md:34–46 RPS 表行 1 (Ping 64254) / 行 3 (authed 58406)

面试 Q&A（上）

Q1：access 24h / refresh 10d 这两个数字怎么定?
A：access 短限泄露损失，refresh 长覆盖周末回流，10d 是留存与安全的折中。

Q2：RBAC 30s 内存缓存，admin 误授权 30s 不是漏洞吗?
A：常规延迟可接受。关键路径 `InvalidateRoleCache(userId)` 显式失效，提权 / 降权立即生效。

Q3：登出后老 token 还能用 24h 怎么办?
A：token 仅控访问不直接控钱，大额二次验证。路线图加 Redis 黑名单 + `token_ver`。

Q4：bcrypt cost=12 怎么选？怎么无痛升 14?
A：250ms 在 300ms 感知阈值内，穷举不现实。登录成功路径探测旧 hash 顺便 re-hash，90 天自然升完。

Q5：登录接口怎么扛 15k RPS 撞库?
A：IP + 账号双维度滑动窗口 + 5 次失败强制验证码 + 30min 冻结，bcrypt 不消耗在冻结账号上。

Q6：admin bootstrap 一次性接口怎么防重?
A：`count(*) WHERE role='admin'`，>0 直接返回错。并发加 advisory lock 兜底。

面试 Q&A（下）

Q7：merchant 没独立 group 为什么不阻塞上线？
A：单店家场景 user 即 merchant。多店家才需 group + DAO 层 shop_id 双层。

Q8：业务码 30001 vs 30002 用户视角一样，区分有何意义？
A：用户都重登，客服区分” 安全事件 vs 长闲置”，比例异常触发审计。

Q9：客服强制下线如何实现？多端怎么隔离？
A：users 加 `token_ver` 签进 claims，改密码 / 下线时 ++。多端按 `client_type` 分桶存 `token_ver_map`。

Q10：为什么 gomall 选 JWT 不选 session？
A：零 RTT + 无状态扩展 + 多端 header 无跨域。代价是失效慢，`token_ver` 兜底。银行业务选 session 更合理。

Q11：第三方登录怎么接入？
A：OAuth 拿 `external_id` → 查/建 `user_oauth` → 复用 GenerateToken。后续仍走 AuthMiddleware。

Q12：100k QPS 鉴权怎么扩？瓶颈在哪？
A：HS256 + sync.Map 单机 58k RPS，100k 扩 4 实例够用，瓶颈始终在下单 / 支付 / 库存。

代码位置一览

路由 / authed / admin *routes/router.go:34–45, 47–48, 146, 148–155*

AuthMiddleware / SetToken *middleware/jwt.go:16–64, 66–73*

双 token 续期 / EmailClaims *pkg/utils/jwt/jwt.go:25–52, 67–93, 95–134*

RequireRole + 30s 缓存 / Invalidate *middleware/rbac.go:22–45, 48–86, 88–91*

admin bootstrap / promote *service/admin.go:47–62, 66–80*

bcrypt cost=12 / Role *repository/db/model/user.go:23, 28, 34–48*

业务码 / 时长常量 *pkg/e/code.go:35–58；consts/user.go:31–32*

滑动窗口 / Redis 客户端 *repository/cache/ratelimit/sliding_window.go；config/redis.go*

压测数据来源 *stressTest/REPORT.md:34–46*

路线图 *user.go 加 TokenVer / TokenVerMap；admin.go 加 ForceLogout + AuditMiddleware；oauth.go* 新建